Clean Code

Lire, comprendre et documenter du code

Session 1 / 2

Master 2 Data & Strategy

Dcid CONSULTING

Pourquoi le Clean Code ?

> *"Any fool can write code that a computer can understand. Good programmers write code that **humans** can understand."*
>
> — Martin Fowler

80%

du temps d'un dev est passé à LIRE du code

10x

plus coûteux de corriger un bug en prod vs en dev

6 mois

pour comprendre un code legacy mal documenté

La dette technique

Accumulation de choix techniques sous-optimaux qui ralentissent le développement futur.

Causes

- Code sans documentation
- Variables mal nommées
- Copier-coller au lieu de factoriser
- Corrections rapides ("ça marche")
- Absence de revue de code

Conséquences

- Temps de compréhension ×10
- Bugs récurrents
- Onboarding difficile
- Réticence à modifier le code
- Coût de maintenance explosif

Naviguer dans la documentation

help()

Documentation intégrée Python

```
help(pd.DataFrame.merge)
```

? / ??

Raccourcis Jupyter

```
pd.read_csv?
```

docs.python.org

Documentation officielle

```
pandas.pydata.org/docs
```

Interpréter du code existant

Méthodologie de lecture

1. **Vue d'ensemble** — Structure, imports, classes/fonctions principales
2. **Point d'entrée** — Identifier le main(), la fonction appelée en premier
3. **Flux de données** — D'où viennent les inputs ? Où vont les outputs ?
4. **Blocs logiques** — Nommer mentalement chaque section (chargement, transformation, export)
5. **Anti-patterns** — Repérer le code dupliqué, les magic numbers, le code mort

Anti-patterns à repérer

Magic Numbers

if status == 3: → if status == STATUS_ACTIVE:

Code dupliqué

Mêmes lignes copiées-collées → factoriser en fonction

God Object

Classe qui fait tout → diviser en responsabilités

Code mort

Fonctions jamais appelées → supprimer

Nommage obscur

x, temp, data2 → revenue, cleaned_df

Fonctions géantes

>50 lignes → découper en sous-fonctions

Commenter vs Documenter

Commentaires

QUAND utiliser :
- Expliquer le POURQUOI
- Avertir d'un piège
- TODO / FIXME temporaires

À ÉVITER :
- Paraphraser le code
- Commentaires obsolètes
- Commenter du code mort

Docstrings """

USAGE :
- Début de module/classe/fonction
- Description du comportement
- Paramètres (Args)
- Valeurs de retour (Returns)
- Exceptions (Raises)
- Exemples d'utilisation

→ *Accessible via help()*

Conventions de docstrings

Style NumPy (recommandé pour data science)

```
def calculate_revenue(transactions, tax_rate=0.2):
    """
    Calcule le revenu total après application de la taxe.

    Parameters
    ----------
    transactions : pd.DataFrame
        DataFrame contenant les colonnes 'amount' et 'quantity'.
    tax_rate : float, optional
        Taux de taxe à appliquer (default: 0.2).

    Returns
    -------
    float
        Revenu total après taxe.

    Examples
    --------
    >>> df = pd.DataFrame({'amount': [100, 200], 'quantity': [1, 2]})
    >>> calculate_revenue(df, tax_rate=0.1)
    450.0
    """
```

Structurer un projet data

```
📁 mon_projet/
├── 📁 data/
│   ├── raw/          # Données brutes
│   └── processed/    # Données nettoyées
├── 📁 notebooks/
│   └── exploration.ipynb
├── 📁 src/
│   ├── __init__.py
│   ├── data_loader.py
│   ├── transformations.py
│   └── utils.py
├── 📁 tests/
├── requirements.txt
└── README.md
```

Séparation des responsabilités

Un fichier = une fonction principale

Imports relatifs

from .utils import clean_data

Configuration centralisée

Chemins, constantes dans config.py

__init__.py

Expose l'API publique du module

Récapitulatif Session 1

- ✓ Le clean code réduit la dette technique et facilite la maintenance
- ✓ Utiliser help(), ? et la documentation officielle
- ✓ Lire le code méthodiquement : structure → flux → détails
- ✓ Repérer les anti-patterns : magic numbers, duplication, nommage
- ✓ Documenter avec des docstrings NumPy style
- ✓ Organiser son projet avec une structure claire

A suivre : Session 2 - Audit et analyse propre avec Pandas